```
import itertools
import re

def eval_expression(expr, model):
    """Evaluates a logical expression string given a truth assignment dictionary."""
    # Convert to uppercase first
    safe_expr = expr.upper()

    # Clean up spaces around operators to avoid word boundary issues
    safe_expr = safe_expr.replace("¬", " ~ ")
    safe_expr = safe_expr.replace("&", " & ")
    safe_expr = safe_expr.replace("V", " V ")
    safe_expr = safe_expr.replace("⇔", " <=> ")

    # Handle implicit multi-word operators before translating to Python lower case
    safe_expr = safe_expr.replace("NOT ", " not ")
    safe_expr = safe_expr.replace("~", " not ")
    safe_expr = safe_expr.replace(" AND ", " and ")
    safe_expr = safe_expr.replace(" & ", " and ")
    safe_expr = safe_expr.replace(" OR ", " or ")
    safe_expr = safe_expr.replace(" V ", " or ")
    safe_expr = safe_expr.replace("<=>", " == ")
    safe_expr = safe_expr.replace("==", " == ")

    # Inject variable values from the model map into local scope for eval
    return bool(eval(safe_expr, {}, model))

def parse_symbols(expr1, expr2):
    """Extracts all unique alphabetical variable symbols, ignoring keywords."""
    all_text = expr1.upper() + " " + expr2.upper()
    found = re.findall(r'\b[A-Z]\b', all_text)
    # Exclude 'V' since it's our OR operator symbol
    return set(sym for sym in found if sym != 'V')

# --- Interactive Main Loop ---
if __name__ == "__main__":
    print("=" * 60)
    print("      PL TRUTH-TABLE ENTAILMENT ENUMERATION TOOL (FIXED)")
    print("=" * 60)
    print("Supported Operators:")
    print("  Negation (NOT):     ¬  or  ~  or  not")
    print("  Conjunction (AND):  &  or  and")
    print("  Disjunction (OR):   V  or  or")
    print("  Biconditional (IFF):⇔  or  <=>")
    print("-" * 60)

    kb_input = input("Enter Knowledge Base (KB / premise) sentence: ").strip()
    alpha_input = input("Enter Query (α / conclusion) sentence: ").strip()

    symbols = sorted(list(parse_symbols(kb_input, alpha_input)))

    if not symbols:
        print("\n❌ Error: No valid single-letter variables detected.")
    else:
        print(f"\nDetected Variables: {symbols}")
        print("\n--- Generating Truth Table ---")

        headers = symbols + [f"KB", f"Query", "Row Status"]
        print(" | ".join(f"{h:<12}" for h in headers))
        print("-" * (15 * len(headers)))

        all_combinations = list(itertools.product([True, False], repeat=len(symbols)))
        entails = True

        for combo in all_combinations:
            model = dict(zip(symbols, combo))

            kb_val = eval_expression(kb_input, model)
            alpha_val = eval_expression(alpha_input, model)

            status = "Valid (KB is False)"
            if kb_val:
                if alpha_val:
                    status = "✅ Valid Match"
                else:
                    status = "❌
```

```
                    status = "❌ FAILURE"
                    entails = False

            row_str = [str(model[sym]) for sym in symbols] + [str(kb_val), str(alpha_val), status]
            print(" | ".join(f"{item:<12}" for item in row_str))

        print("-" * (15 * len(headers)))
        if entails:
            print(f"\n🎉 RESULT: True! '{kb_input}' ENTAILS '{alpha_input}'.")
        else:
            print(f"\n❌ RESULT: False! '{kb_input}' DOES NOT ENTAIL '{alpha_input}'.")
```

```
============================================================
      PL TRUTH-TABLE ENTAILMENT ENUMERATION TOOL (FIXED)
============================================================
Supported Operators:
  Negation (NOT):     ¬  or  ~  or  not
  Conjunction (AND):  &  or  and
  Disjunction (OR):   V  or  or
  Biconditional (IFF):⇔  or  <=>
------------------------------------------------------------
Enter Knowledge Base (KB / premise) sentence: (not Q or P) & (not P or not Q) & (Q V R)
Enter Query (α / conclusion) sentence: R

Detected Variables: ['P', 'Q', 'R']

--- Generating Truth Table ---
P            | Q            | R            | KB           | Query        | Row Status
-----------------------------------------------------------------------------------------
True         | True         | True         | False        | True         | Valid (KB is False)
True         | True         | False        | False        | False        | Valid (KB is False)
True         | False        | True         | True         | True         | ✅ Valid Match
True         | False        | False        | False        | False        | Valid (KB is False)
False        | True         | True         | False        | True         | Valid (KB is False)
False        | True         | False        | False        | False        | Valid (KB is False)
False        | False        | True         | True         | True         | ✅ Valid Match
False        | False        | False        | False        | False        | Valid (KB is False)
-----------------------------------------------------------------------------------------

🎉 RESULT: True! '(not Q or P) & (not P or not Q) & (Q V R)' ENTAILS 'R'.
```